□ Python Notes (Basic + Core) — With Short Descriptions

📌 1. Variables & Data Types

Description: Used to store and represent data in memory. Python is dynamically typed, so type is decided at runtime.

```
x = 10
type(x)
```

📌 2. Strings

Description: Immutable sequence of characters used for text handling.

```
s = "hello"
s[::-1]
```

Functions:

```
s.lower(), s.upper(), s.strip(), s.replace(), s.split(), s.find()
```

📌 3. Lists

Description: Ordered, mutable collection used to store multiple values.

```
li = [1,2,3]
```

Functions:

```
li.append(), li.insert(), li.remove(), li.pop(), li.sort(), li.reverse()
```

📌 4. Tuple

Description: Ordered, immutable collection (cannot be modified after creation).

```
t = (1,2,3)
```

Functions:

```
t.count(), t.index()
```

📌 5. Set

Description: Unordered collection with unique elements (no duplicates).

```
s = {1,2,3}
```

Functions:

```
s.add(), s.update(), s.remove(), s.discard(), s.pop()
```

📌 6. Dictionary

Description: Stores data in key-value pairs with fast access using hashing.

```
d = {"a":1}
```

Functions:

```
d.get(), d.keys(), d.values(), d.items(), d.update(), d.pop(),
d.setdefault()
```

📌 7. Control Flow

Description: Used to control program execution using conditions and loops.

```
if, else, elif
for, while
```

📌 8. Functions

Description: Reusable block of code that performs a specific task.

```
def f(x): return x
```

📌 9. List Comprehension

Description: Compact way to create lists in a single line.

```
[x for x in range(10)]
```

📌 10. Slicing

Description: Extracts a part of sequence using `[start:end:step]`.

```
li[::-1]
```

📌 11. Modules

Description: Python files containing reusable code.

```
import math
```

📌 12. File Handling

Description: Used to read and write data to files.

```
open("file.txt","r")
```

📌 13. Pickle

Description: Converts Python objects into byte stream and vice versa.

```
pickle.dump(), pickle.load()
```

📌 14. JSON

Description: Text-based format used for data exchange between systems.

```
json.dumps(), json.loads()
```

📌 15. Date & Time

Description: Used to work with current date, time, and formatting.

```
datetime.now(), strftime()
```

📌 16. Random Module

Description: Generates random numbers and selections.

```
random.randint(), random.choice()
```

📌 17. Mini Project (Rock-Paper-Scissors)

Description: Practical implementation combining logic, loops, conditions, and data structures.

📌 18. Stack

Description: Linear data structure that follows **LIFO (Last In First Out)** principle.

👉 Last inserted element is removed first

🔹 Operations

```
push → append()
pop  → pop()
peek → li[-1]
```

💻 Example

```
li = []

li.append(10)   # push
li.append(20)

li.pop()        # removes 20
```

📌 Key Points

- Top = last element
- Used in:

    - recursion
    - undo operations
- Implemented using list or deque

📌 19. Queue

Description: Linear data structure that follows **FIFO (First In First Out)** principle.

👉 First inserted element is removed first

🔹 Operations

```
enqueue → append()
dequeue → pop(0)
```

💻 Example

```
li = []

li.append(10)   # enqueue
li.append(20)

li.pop(0)       # removes 10
```

📌 Key Points

- Front = first element
- Used in:
    - scheduling
    - buffering
- Better implementation → deque